Experiment 10: Text Summarization

```
!pip -q install networkx nltk transformers torch

import nltk

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

text = """
Text summarization methods can be grouped into two main categories:
Extractive and Abstractive methods.

Extractive Text Summarization is the traditional method developed first.
The main objective is to identify the significant sentences of the text
and add them to the summary. The summary contains exact sentences
from the original text.

Abstractive Text Summarization is a more advanced method. It identifies
important sections, interprets the context and reproduces the information
in a new way. The sentences in the summary are generated rather than
simply extracted from the original text.

Gensim is a Python library used for natural language processing.
Its older summarization API uses the TextRank algorithm. TextRank is
an extractive summarization technique that scores sentences and selects
the most important ones.

The summary length can be controlled using a ratio or a word count.
The ratio represents the proportion of the original text to retain,
while word count specifies the approximate number of words.
"""

print("Input text:\n")
print(text)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Input text:


Text summarization methods can be grouped into two main categories:
Extractive and Abstractive methods.

Extractive Text Summarization is the traditional method developed first.
The main objective is to identify the significant sentences of the text
```

```
and add them to the summary. The summary contains exact sentences
from the original text.

Abstractive Text Summarization is a more advanced method. It identifies
important sections, interprets the context and reproduces the information
in a new way. The sentences in the summary are generated rather than
simply extracted from the original text.

Gensim is a Python library used for natural language processing.
Its older summarization API uses the TextRank algorithm. TextRank is
an extractive summarization technique that scores sentences and selects
the most important ones.

The summary length can be controlled using a ratio or a word count.
The ratio represents the proportion of the original text to retain,
while word count specifies the approximate number of words.

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

A. TextRank summarization

```
import re
import networkx as nx

from nltk.tokenize import sent_tokenize
from nltk.corpus import stopwords

def textrank_summary(text, sentence_count=3):
    sentences = sent_tokenize(text)

    if not sentences:
        return ""

    if len(sentences) <= sentence_count:
        return " ".join(sentences)

    stop_words = set(stopwords.words("english"))

    # Create a set of meaningful words for each sentence
    sentence_words = []

    for sentence in sentences:
        words = re.findall(r"\b[a-zA-Z]+\b", sentence.lower())
        meaningful_words = {
            word for word in words if word not in stop_words
        }
        sentence_words.append(meaningful_words)

    # Create a sentence similarity graph
    graph = nx.Graph()
```

```
    graph.add_nodes_from(range(len(sentences)))

    for i in range(len(sentences)):
        for j in range(i + 1, len(sentences)):
            common = len(sentence_words[i] & sentence_words[j])
            total = len(sentence_words[i] | sentence_words[j])

            if total > 0 and common > 0:
                similarity = common / total
                graph.add_edge(i, j, weight=similarity)

    scores = nx.pagerank(graph, weight="weight")

    # Select top-scoring sentences and retain their original order
    selected = sorted(
        sorted(scores, key=scores.get, reverse=True)[:sentence_count]
    )

    return " ".join(sentences[i] for i in selected)


summary = textrank_summary(text, sentence_count=3)

print("Summary using TextRank:\n")
print(summary)
```

```
Summary using TextRank:

Extractive Text Summarization is the traditional method developed first. The
from the original text. Abstractive Text Summarization is a more advanced met
```

B. GPT-2 text generation

```
from transformers import pipeline, set_seed

set_seed(42)

generator = pipeline(
    "text-generation",
    model="gpt2"
)

# Use a short input prompt to fit the model's context
prompt = text[:1000] + "\n\nTL;DR:"

result = generator(
    prompt,
    max_new_tokens=60,
    do_sample=False,
    pad_token_id=generator.tokenizer.eos_token_id,
```

```
    truncation=True
)

generated_text = result[0]["generated_text"]

# Display only the newly generated portion
generated_summary = generated_text[len(prompt):].strip()

print("GPT-2 generated text:\n")
print(generated_summary)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
config.json: 100%  665/665 [00:00<00:00, 59.6kB/s]
model.safetensors: reconstructing file: 100%  548MB / 548MB, 43.0MB/s
model.safetensors: downloading bytes:  474MB, 43.7MB/s
Loading weights: 100%  148/148 [00:00<00:00, 2512.43it/s]
generation_config.json: 100%  124/124 [00:00<00:00, 12.8kB/s]
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 2.13kB/s]
vocab.json: 100%  1.04M/1.04M [00:00<00:00, 9.70MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 15.4MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 26.0MB/s]
GPT-2 generated text:

The ratio is the ratio of the original text to the

original text.

The word count is the number of words in the original text.

The word count is the number of words in the summary.

The word count is the number of words in the summary.
```